



Secured Application Development

Secured Application Development

Transport and Encryption



Cybersecurity



Transport and Encryption

Dr. Abdullmalek Alqobaty

**Associated professor
Computer Engineering**

Secured Application Development

Transport and Encryption



Transport and Encryption

Transport and Encryption



Transport and Encryption

Transport and Encryption

In this lecture, we look at the most fundamental building block of web applications: **the HTTP protocol**. This is the protocol browsers and web servers use to communicate with each other.

- Most web applications use the encrypted version of HTTP, called HTTPS. So that we can understand this, as well as some topics in future, we look in some detail at encryption methods before learning how they are applied in the HTTPS protocol.
- Encryption techniques fall into two categories: symmetric and public key. Both are covered in this lecture.
- We also look at TLS/SSL certificates, which are an essential component of HTTPS.
- As HTTP and encryption are so fundamental to web application security, we will use our hands-on environment to explore the techniques covered in this lecture.



The Hypertext Transfer Protocol

To understand how to build secure web applications, we must understand how web clients and servers communicate.

- ❑ Clients send requests to servers, and servers send back responses. (**Client-Server model**)
 - The main protocol for web requests and responses is the Hypertext Transfer Protocol (HTTP) or
 - Encrypted version, HTTPS (the “S” stands for secure).
- ❑ The most used version is 1.1 and is described by an RFC 2616, which can be found at: www.rfc-editor.org/rfc/rfc2616.txt



Transport and Encryption

Requests and Responses

- ❑ HTTP is a simple request-response protocol running on port 80.
 - A client (e.g., a web browser) sends a request to a server.
 - The server sends back a response.
- ❑ HTTP is stateless: each request and response is self-contained, and no data is stored in between requests.



Requests and Responses

Example HTTP request

GET / HTTP/1.1

Host: localhost

- ☐ **GET** is the method, requesting that the server sends client a resource.
- ☐ After **GET** is the URI of the resource being requested, **/ in this case**. This is the part of the URL after the hostname (and port number, if that is in the URL).
- ☐ The third parameter is the protocol **HTTP version 1.1 in this case**; the resource is being requested over **Http**.
- ☐ The following line specifies the host **the resource is being requested from, localhost**.
- ☐ A colon followed by a port number can be appended to the hostname. **If it is not, as in our case, it defaults to 80.**



Transport and Encryption

GET Request Example (Retrieving a resource):

Code

```
GET /index.html HTTP/1.1
```

```
Host: www.example.com
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,  
like Gecko) Chrome/120.0.0.0 Safari/537.36
```

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
```

```
Accept-Language: en-US,en;q=0.
```

```
Connection: keep-alive
```

Explanation:

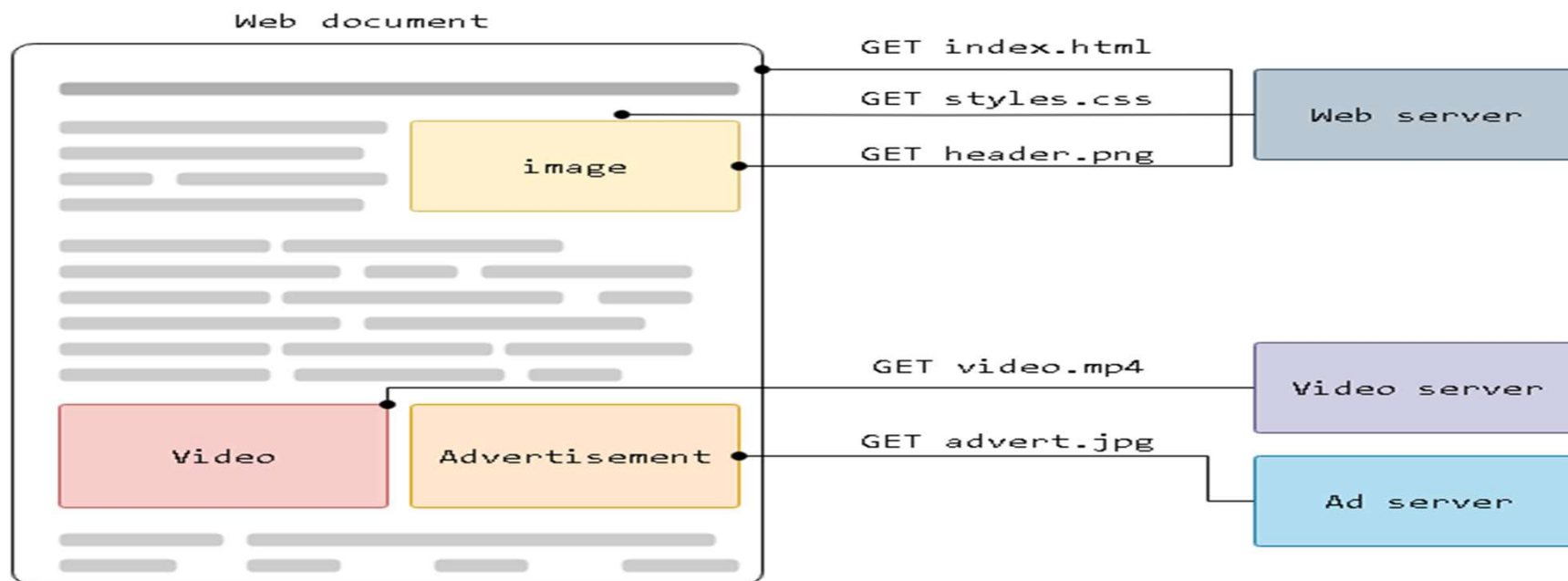
- **GET /index.html HTTP/1.1:** The request line specifies the HTTP method (GET), the target resource (/index.html), and the HTTP protocol version (HTTP/1.1).
- **Host:** www.example.com: The Host header indicates the domain name of the server being targeted.
- **User-Agent, Accept, Accept-Language, Connection:** These are common headers providing information about the client and its preferences.



Transport and Encryption

Requests and Responses

- ❑ HTTP is a client-server protocol for fetching resources. Requests are initiated by the client, usually the Web browser. A complete web document is typically constructed from resources (text, layout instructions, images, videos, scripts, etc..

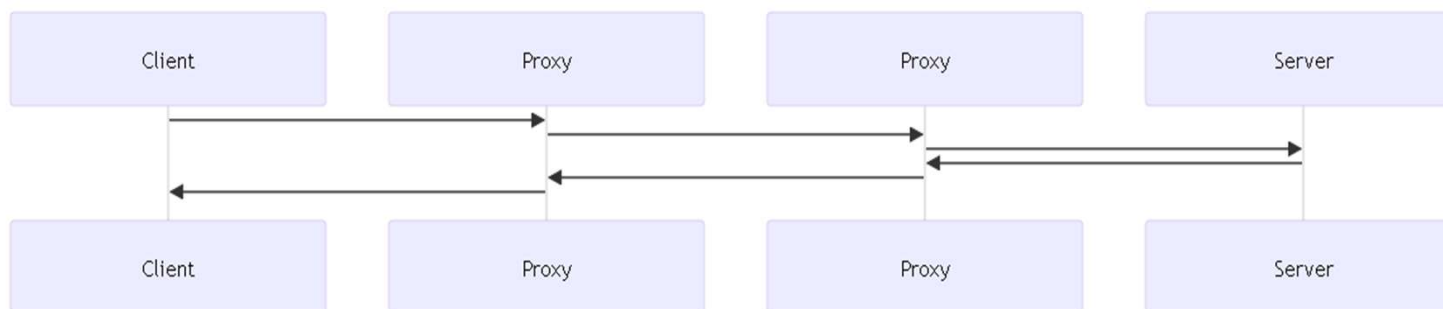




Transport and Encryption

Requests and Responses

- ❑ Each request is sent to a server to handle it and provides an answer called the **response**. Between the client and the server **there are numerous entities, collectively called proxies**, which perform different operations and act as gateways or **caches**, for example.
- ❑ A **proxy server is an intermediate program or computer used when navigating through different networks of the Internet. They facilitate access to content on the World Wide Web.**
- ❑ A proxy intercepts requests and serves back responses; it may forward the requests or not. It may modify it.





Transport and Encryption

Requests and Responses

- ☐ **We use Get HTTP method.** Other supported methods in HTTP/1.1 are listed below.
Unless writing REST APIs, you will commonly use GET and POST.

Method	Purpose
GET	Retrieve the named resource from the server.
HEAD	Same as GET but returns the status and headers only, not the body.
OPTIONS	Ask the server which HTTP methods it supports for the named resource. Does not return the actual resource.
POST	Create a subresource for the named resource on the server. Commonly also used for submitting form data.
PUT	Replace the name resource with a new one, from the data in the body.
DELETE	Delete the named resource from the server.
CONNECT	Establish a connection that can be used as a tunnel. Used by proxies.
PATCH	Partial update of the named resource. The body only needs to contain the fields that have changed.
TRACE	Instruct the server to send the request back to the client in the response. Used for diagnostic purposes.



Transport and Encryption

Requests and Responses

- ☐ Other headers can optionally follow, one line each with the form
 - ☑ Header-Name: value
- ☐ For some methods (e.g., POST), a body follows the headers, after a blank line.
- ☐ A Content-Type header is needed to tell the server what format the body is in (see the next section).
- ☐ If the server is listening on the requested port, the client will receive a response. It has a similar format to the request.

- ☑ An example is

HTTP/1.1 200 OK

Date: Thu, 07 Oct 2021 07:27:47 GMT

Server: Apache/2.4.41 (Ubuntu)

...

Content-Type: text/html; charset=utf-8

<!DOCTYPE html>

<html lang="en">

...

</html>



Transport and Encryption

Requests and Responses

Examples

Request

```
GET /articles/123 HTTP/1.1
Host: www.example.com
Accept: text/html
```

Response

```
HTTP/1.1 200 OK
Content-Type: text/html
<html>
  <body>
    <h1>Title</h1>
    <p>Body</p>
  </body>
</html>
```

Request

```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/json
{
  "username": "user123",
  "password": "password123"
}
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "message": "Login is Done"
}
```

Request

```
PUT /articles/123 HTTP/1.1
Host: www.example.com
Content-Type: application/json
{
  "title": "New title",
  "content": "New contents"
}
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "message": "Update successfully"
}
```



Transport and Encryption

Requests and Responses

Examples

Request

PATCH /users/123 HTTP/1.1
Host: api.example.com
Content-Type: application/json-patch+json
Authorization: Bearer YOUR_ACCESS_TOKEN

```
[  
  { "op": "replace", "path": "/email", "value":  
    "newemail@example.com" },  
  { "op": "add", "path": "/phoneNumbers/-",  
    "value": "555-123-4567" }  
]
```

Response

HTTP/1.1 204 No Content
Content-Location: /users/123
ETag: "some-new-etag-value"

Request

DELETE /articles/123 HTTP/1.1
Host: www.example.com

Response

HTTP/1.1 204 No Content



Requests and Responses

Response Codes

HTTP response codes are numbers between 100 and 599. They are grouped as follows:

- 1xx: Informational codes
- 2xx: Successful completion of a request
- 3xx: Redirection to another resource
- 4xx: A client error occurred
- 5xx: A server error occurred



Requests and Responses

Response Codes

- ☐ **200 OK:** Normal successful response when a body is also returned.
- ☐ **201 Created:** Successful response when an object is created and its ID is returned.
- ☐ **204 No Content:** Successful response when no body is returned.
- ☐ **301 Moved Permanently:** Permanent redirect (e.g., HTTP to HTTPS). The client may cache this and request the new URL directly in future.
- ☐ **302 Found:** Redirection that is only temporary, for example, to a new page after processing a form. It is not cached by the browser unless enabled by other headers.
- ☐ **403 Forbidden:** Error status when client does not have permission to access the resource (e.g., when not logged in).
- ☐ **404 Not Found:** Error status when the URL or resource does not exist.
- ☐ **405 Method Not Allowed:** Error status when the URL is valid but not the HTTP method (e.g., GET requested but only POST supported).
- ☐ **500 Internal Server Error:** An error occurred on the server (e.g., a code exception was thrown).

Refer to the book for full code list.



Symmetric and Public Key Cryptography

Types of Encryption

- ❑ **Encryption is a vital tool in securing web applications.** It uses a digital key to render data unreadable without decoding them using a corresponding decryption key.
- ❑ We call the original, unencrypted data the **plaintext** and the encrypted data **cipher text**.
- ❑ The two classes of encryption are:
 - **symmetric-key encryption** (secret-key encryption)
 - **public-key encryption:**



Transport and Encryption

Types of Encryption

- ❑ **symmetric-key encryption** (secret-key encryption): the same key is used for encryption and decryption.
- ❑ **public-key encryption**: Public-key encryption uses two keys. The sender uses the recipient's public key to encrypt data, and the recipient uses their private key to decrypt them. **Note that: The public key can be derived from the private key but not vice versa (at least not easily).**
 - An advantage of public-key encryption: is that the encryption key need not be kept secret.
 - The two disadvantages are:
 - Sending the same data to multiple recipients requires encrypting them with each recipient's public key separately
 - It slower than symmetric-key encryption.



Symmetric-Key Algorithms

- ❑ Symmetric-key algorithms can be block or stream ciphers.
 - Block ciphers work on a fixed-length block of data. Data are split into chunks of the required size, and the last chunk is padded if necessary.
 - Stream ciphers encode data in a continuous stream rather than in blocks.
- ❑ The security of an encryption algorithm is a function of the **length of its key**. Longer keys are harder to guess, but encryption and decryption take longer. The lengths are measured in bits.



Symmetric-Key Algorithms

- ❑ The most common block cipher symmetric-key algorithms are **Advanced Encryption Standard (AES)** and S is newer and considered more secure. It comes in 128-, 192-, and 256-bit versions (commonly abbreviated as AES-128, AES-192, and AES-256).
- ❑ AES has been adopted by the U.S. government. It supersedes the **Data Encryption Standard (DES)**
- ❑ Other block ciphers include (**Rivest Cipher**) **RC5 and RC6**.
- ❑ **RC4 is a stream cipher. (128 bits or greater).**



Public-Key Algorithms

- ❑ In public-key encryption, encryption and decryption keys are different. The encryption (public) key needs not be kept secret as the decryption (secret) key that cannot be derived from it.
- ❑ The best-known public-key encryption algorithm is **Rivest-Shamir-Adleman (RSA)**. It relies on the fact that finding prime factors of very large numbers is difficult.
- ❑ An attraction of RSA is that different key lengths can be used. Longer keys means more security but more computation time. Keys are typically 512, 1024, 2048, or 4096 bits. (2048- 2030 is good)
- ❑ Another public-key popular algorithm is **Elliptic Curve Digital Signature Algorithm (ECDSA)**. Its advantage is that, compared with RSA, a shorter key length, and therefore faster computation, can achieve the same level of security. (224 bits or greater)



Hashing

- ❑ Hashing also makes data unreadable. But, hashing is a one-way function. It is useful for storing passwords. **The advantage is that if the password file is compromised, users' passwords are not.**
- ❑ **Hashes** are often used to create a **digest** or **checksum** of plaintext, which is used to confirm integrity.
- ❑ As hashes are shorter than the plaintext, they are not necessarily unique. If two sets of data result in the same hash, it causes a collision. In practice, this rarely causes problems.
 - **A good hashing algorithm has high entropy (a lot of information is coded in a small number of characters).**
- ❑ The most common hashing algorithms are **Secure Hashing Algorithm (SHA)** and **Message Digest 5 (MD5)**.
 - **SHA supports a range of bit lengths, with 256 and 384 the most common.**



Base64 Encoding

- Encryption and hashing algorithms produce binary data, even if the plaintext is ASCII. **This is problematic if a protocol requires an ASCII representation.**
- One solution is to represent the bytes as hexadecimal characters. **Each byte therefore takes two bytes to encode**, as two hexadecimal characters are needed to represent an 8-bit number.
- Base64 is a more efficient encoding algorithm that converts binary data to text by encoding **every three 8-bit bytes as four 6-bit bytes**. **If the plaintext length is not a multiple of three, it is padded.**
 - ☑ Thus, a Base64-encoded string is a third longer (or a little more if padded) than the original binary data.
- Base64 is often used in preference over hexadecimal notation as fewer bytes are required.



Transport and Encryption

Digital Signatures

- Digital signatures differ from both encryption and hashing. **They enable a recipient of data to verify its integrity.**
 - ☑ **The sender encrypts data using their private key, not the recipient's public key. This forms a signature that is sent along with the plaintext.**
 - ☑ **The recipient decrypts the signature using the sender's public key and checks the plaintext matches the version in the signature.** For secrecy, the plaintext or the entire message can also be encrypted with the recipient's public key.
 - ☑ **This results in a signature that grows with plaintext size. To obtain a short, fixed-length signature, a hashing algorithm such as SHA is first applied to the plaintext, and the hash is signed rather than the full plaintext data.**



Digital Signatures advantages

- The advantages of digital signatures over hashes are as follows:
 - ☑ Only a holder of the secret key can create the signature, meaning the recipient can validate the authenticity of the sender.
 - ☑ The public key only has to be sent once, not for each message as a checksum would. It can be sent on a side channel.



Key Exchange

- The advantage of Public-key encryption is that only public keys need to be transmitted, keeping the private keys secret.
- However, symmetric-key algorithms are much faster. For this reason, public-key encryption such as RSA is often used to securely negotiate a symmetric key.
 - ▣ After the exchange, symmetric-key encryption is used to exchange data.
- Diffie-Hellman Key Exchange (DH) is an algorithm for exchanging keys using a public-private key pair.
 - ▣ Its unique feature is that the symmetric key is never transmitted and is a product of both parties' public and private keys.



Transport and Encryption

Key Exchange

To see how DH works, say **Bob** and **Alice** wish to **communicate** with each other using a symmetric key.

1. **The communicating parties** (we'll call them Bob and Alice) agree on two large numbers: **p** (is a prime number) and **g** (is an integer) such that $1 < g < p$.
2. **Bob** chooses a random large integer **r** and performs the following calculation: **$R = g^r \bmod p$**
3. **Alice** chooses a random large integer **s** and performs the following calculation: **$S = g^s \bmod p$**
4. **Bob** sends **R** to **Alice** and **Alice** sends **S** to **Bob**.
5. **Bob** then performs the following calculation: **$K = S^r \bmod p$**
6. **Alice** then performs the following calculation: **$K = R^s \bmod p$**

At this point, Bob and Alice both have the same value, K , and can use this as a secret key.

If an eavesdropper spied on the exchange, they would see **R** and **S** . They may even see **q** and **g** . However, without **S^r** and **R^s** , neither of which is transmitted, **the eavesdropper would be unable to calculate K .**



Transport and Encryption

Key Exchange

1. The communicating parties (we'll call them **Bob** and **Alice**) agree on two large numbers: **p** (is a prime number) and **g** (is an integer) such that $1 < g < p$. **P=31, g=10**

2. **Bob** chooses a random large integer **r** and performs the following calculation: **r=8**

$$R = g^r \bmod p = 10^8 \bmod 31 = 14$$

1. **Alice** chooses a random large integer **s** and performs the following calculation: **s=9**

$$S = g^s \bmod p = 10^9 \bmod 31 = 16$$

4. **Bob** sends **R** to **Alice** and **Alice** sends **S** to **Bob**.

5. **Bob** then performs the following calculation:

$$K = S^r \bmod p = 16^8 \bmod 31 = 4$$

4. **Alice** then performs the following calculation:

$$K = R^s \bmod p = 14^9 \bmod 31 = 4$$



Authentication and Certificates

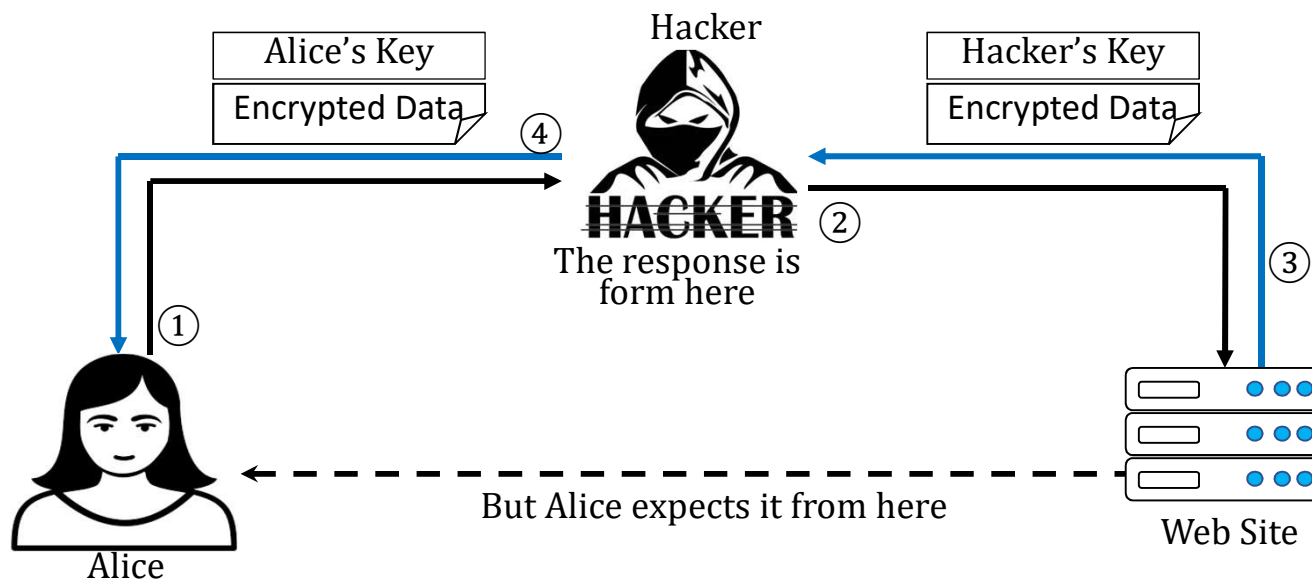
Proving Authenticity

- Public-key encryption provides secrecy but not authenticity.
 - ☒ If Alice can decrypt Bob's message with her private key, she is confident the message is secret and readable only by her. However, she does not know if Bob or someone else sent it.
- In the following Figure,
 1. Alice is attempting to access a website. She makes a request to the website. The request is intercepted by a hacker.
 2. The hacker makes the same request to website using her own key.
 3. The website sends the response to the hacker that decrypts it and then reencrypts it with their own key before forwarding it to Alice.
 4. Alice can decrypt the data but has no way of knowing whether it came from the website or the hacker.



Transport and Encryption

Authentication and Certificates



This is an example of a *man-in-the-middle* attack.



Authentication and Certificates

Proving Authenticity

- The problem is fixed when Alice can get a public key for website that she trusts has not been intercepted by a hacker.
 - ☒ Say there is another organization that Alice trusted (**cert.com**).
 - ☒ **The website asks cert.com to digitally sign a document containing its domain name and public key. Of course, cert.com takes steps to satisfy that the request is coming from the website's owners.**
 - ☒ **If Alice has the public key of cert.com, and the website sends the signed document to Alice before sending the web response, Alice can confirm that the document has come from the website.**
 - ☒ **The signed document is called Secure Sockets Layer(SSL) or Transport Layer Security (TLS) certificate, and cert.com is called a certificate authority (CA).**
 - ☒ **SSL was the basis of HTTPS encryption until it was superseded by TLS, though the terms tend to be used interchangeably.**



Certificate Contents

The certificate usually contains

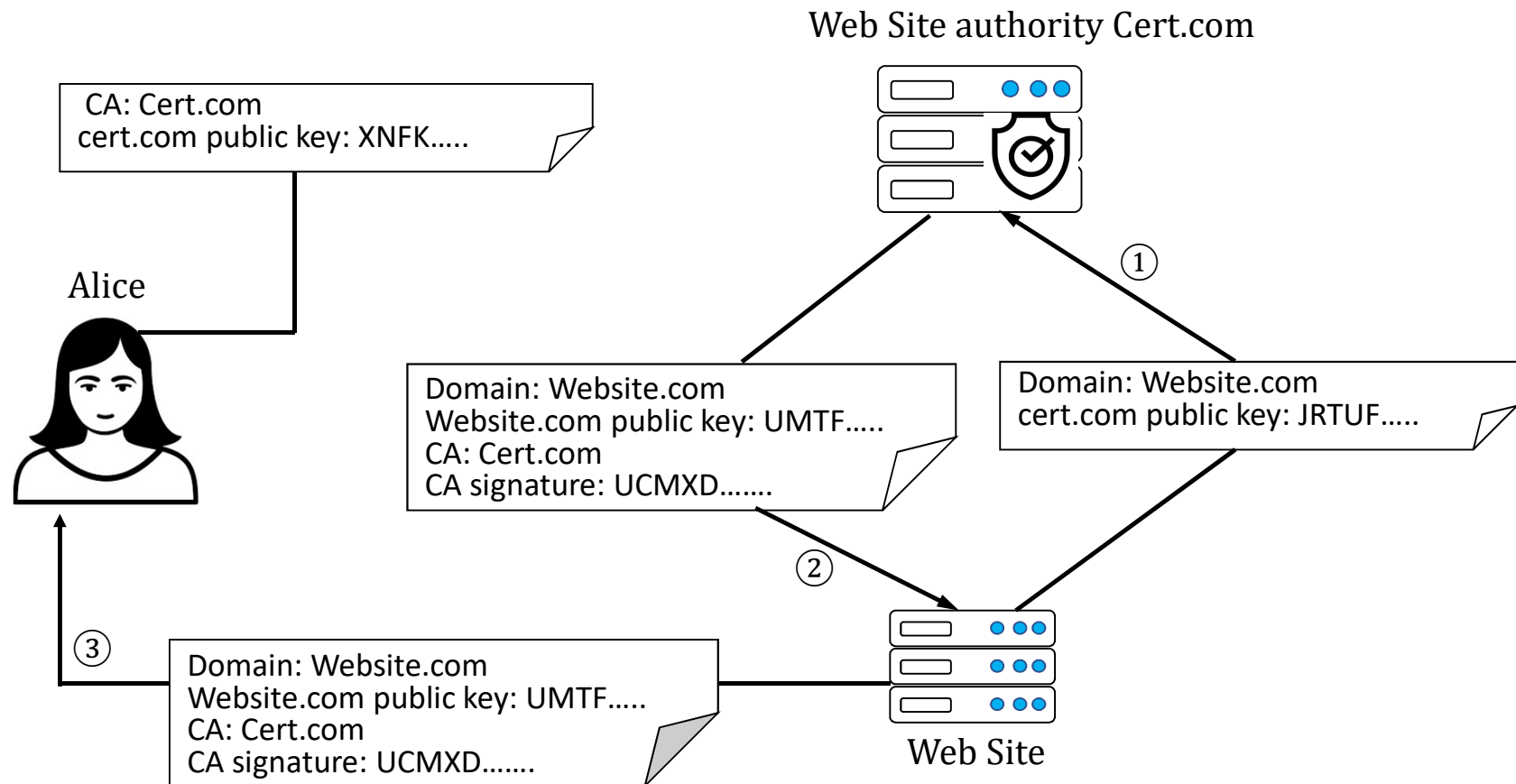
- ☐ **The domain name of the website**
- ☐ **The organization name of the website**
- ☐ **The date of issue**
- ☐ **The date of expiry**
- ☐ **Optionally, the subdomains and additional domain names of the website**
- ☐ **The public key of the website**
- ☐ **The digital signature of the CA**

The process is illustrated in the following Figure below.



Transport and Encryption

Certificate Contents





Transport and Encryption

Certificate Contents

1. The owner of the website requests a certificate from cert.com, sending their public key, domain name, and any other details cert.com needs to confirm the sender is the owner of the domain.
2. Once cert.com has satisfied itself that the requester of the signature is the owner of the website, **it will send back a certificate signed with cert.com's private key.**
 - ☐ It will include an expiry date, between three months' and one years' time.
Website can now send this certificate to Alice's web browser.
3. Alice's web browser is preconfigured with cert.com's public key. **It checks that the domain name in the signature matches the domain name Alice made the request to.**
 - ☐ It also checks the certificate has not expired. If these checks pass, Alice's browser proceeds with the exchange. If not, it warns Alice that the certificate is suspicious.



Transport and Encryption

Certificate Contents

Let us examine what happens in a man-in-the-middle attack when certificates are being sent. This is illustrated in the following Figure.

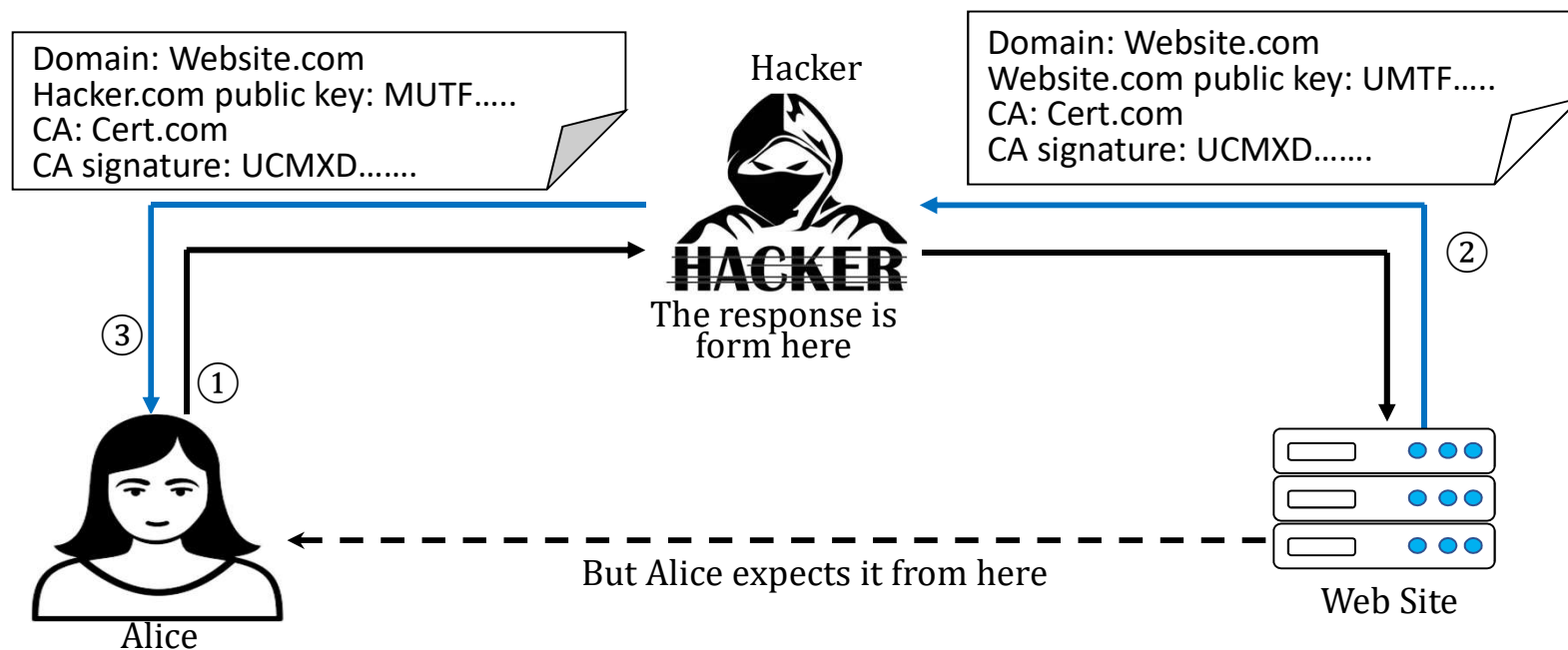
1. Alice makes a request to a website, which is intercepted by the hacker. **The hacker makes her own identical request to the website.**
2. The hacker decrypts the response and reencrypt it and send it to Alice.
3. Alice will not accept the response without a certificate. The hacker has a choice: **send website's certificate or send their own** (for evilhacker.com - cert.com will not issue them with anything else).
4. Say the hacker sends the certificate of the website. **Alice can encrypt a short message using the public key in that certificate, send it to the hacker, and ask that they decrypt it and send it back.** The hacker will be unable to do so because they do not have the website private key.
5. **If hacker simply forward Alice's message to the website.com without decrypting it, they will be unable to decrypt the messages that follow.**



Transport and Encryption

Certificate Contents

6. If the hacker sends their own certificate, for evilhacker.com, as in the diagram, Alice will notice that the domain name is not website and will reject it.





Types of Certificates

- ❑ *To obtain a TLS certificate, you must prove you own domain the certificate is for. For some certificates, you must also prove you own the organization the domain belongs to.*
- ❑ *There are three types of TLS certificate:*
 - ☑ ***Domain validated certificates:** for this certificate, you must only prove you administer the domain name(s) the certificate is for. The CA may validate the domain name by DNS records, Email, or HTTP.*
 - ☑ ***Organization-validated certificates:** for this certificate, you must prove you own the organization. The certificate authority will validate organization's name, physical address, type, and status.*
 - ☑ ***Extended validation certificates** are similar to organization-validated certificates, but more information about the organization is validated, such as, phone number, and business registration number.*
- ❑ *Browsers allow you to view certificates by clicking on the lock icon in the address bar next to the domain name.*



Transport and Encryption

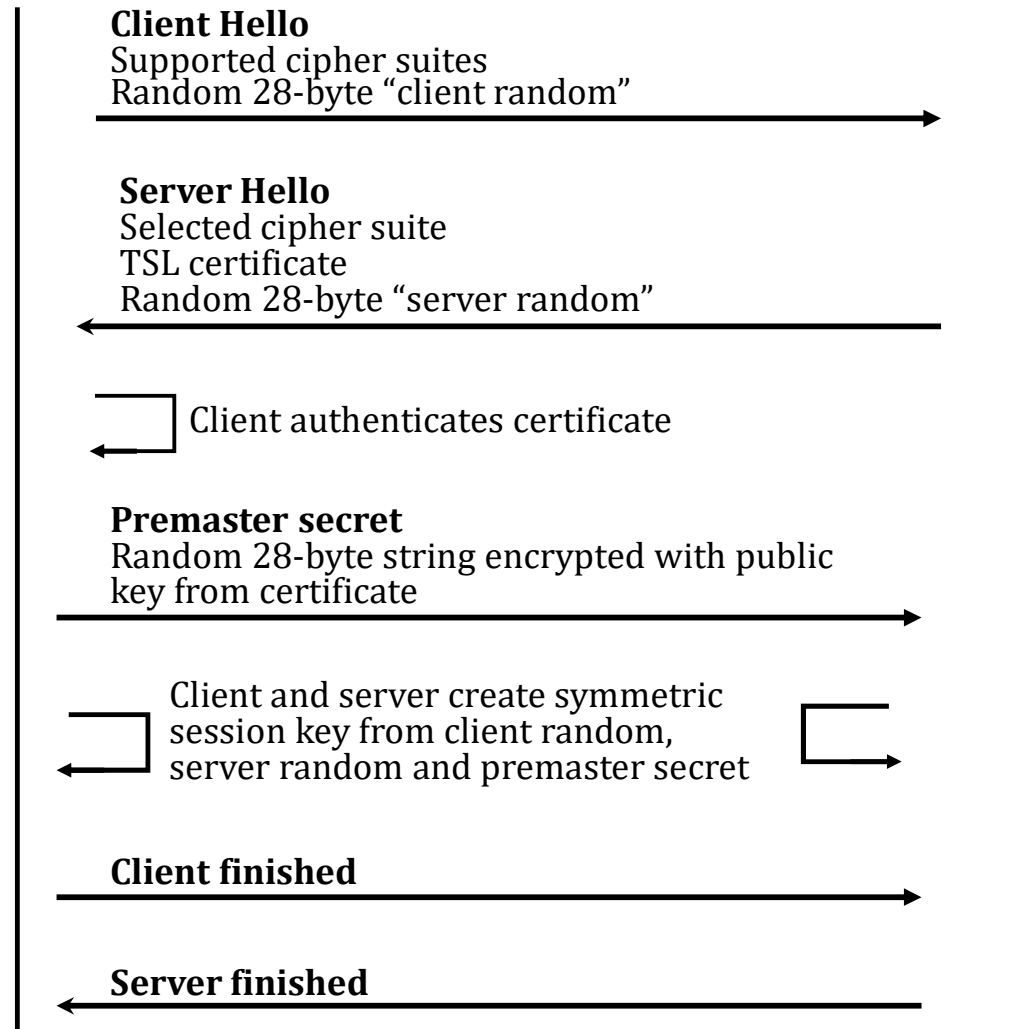
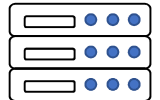
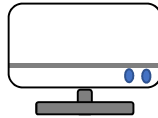
HTTPS: TLS Version 1.2

- HTTPS uses symmetric-key encryption for performance reasons but uses public-key encryption to create the symmetric key and authenticate the server. **The protocol for this is Transport Layer Security (TLS).** It was formerly known as **Secure Sockets Layer (SSL).**
- TLS, encrypts at the transport layer level. This means once a socket connection is established, **everything across that socket is encrypted.**
 - ☒ This includes the HTTP methods (GET, POST, etc.), the GET and POST parameters, HTTP headers, request and response bodies, etc.
 - ☒ The hostname, IP addresses, and port **are not encrypted as they are needed to establish the connection.**
- A TLS connection is established by a TLS handshake: **a multistep process in which the client authenticates the server and together they establish a symmetric key to encrypt the data that follow.** The TLS version 1.2 handshake is illustrated in the Figure below.



Transport and Encryption

HTTPS: TLS Version 1.2





Transport and Encryption

HTTPS: TLS Version 1.2

1. **The client first sends its list of supported cipher suites** (from which the server will choose) and a **28-byte random string (client random)**. This is used later to create the symmetric key.
2. **The server responds with its chosen cipher suite, its TLS certificate, and its 28-byte server random**, used to generate the symmetric key.
3. **The client looks the CA's name up in its list of trusted authorities.** If it is found, it **authenticates the certificate** and, so long as it is valid, proceeds to the next step.
4. **The client generates another 28-byte random string called the premaster secret** and sends this to the server, encrypted with the server's public key.
5. **Now, the server and client now have: The client random, the server random, and the premaster secret**
6. Each can now generate the master secret. **This will be the symmetric session key for encrypting the request and response.**
7. It should now be clear why the hacker in the man-in-the-middle attack cannot simply forward the server's certificate to Alice. **If they did so, Alice and the server would create the session key between them, and the hacker would not be able to decrypt the data.**



Perfect Forward Secrecy

- ❑ With the TLS handshake described previously, a secret key is always created from the same public key of the server.
 - If an attacker were to eavesdrop and copy all the encrypted packets, and later were able to obtain the secret key of the server, they would be able to decrypt all of the previous packets.
- ❑ Servers and browsers often use a variant of Difie-Hellman Key Exchange called Difie-Hellman Ephemeral, or DHE, to avoid this issue.
 - Under DHE, a new, temporary public-private key pair is created by both the client and server with each exchange.



Transport and Encryption

TLS Version 1.3

- ❑ **TLS version 1.3 was introduced in 2018 to improve speed and security compared with version 1.2.** TLS version 1.2 supported 37 cipher suites. TLS 1.3 reduces this number to 5.
- ❑ **TLS 1.3 also speeds up the handshake by reducing the number of steps.** A client guesses the server's preferred cipher suite sends this in the Client Hello along with its set of supported suites. So long as the server is happy with this choice, a step in the handshake is saved. If public keys have to be exchanged, for example, in DHE, this is also bundled in the Client and Server Hellos.
- ❑ **The original server key in the certificate is used to authenticate the server but is not used to create symmetric keys.** This way, even if the attacker obtains the server's private key, they will be unable to decrypt previous data. This is called Perfect Forward Secrecy.
- ❑ **DHE is slower than regular RSA as a new public-private key pair must be created with each exchange.** However, it does protect past data in the event of a private key being disclosed.



Transport and Encryption

TLS Handshake Protocol

Figure 6 shows the initial exchange needed to establish a logical connection between client and server.

- **The exchange can be viewed as having four phases.**

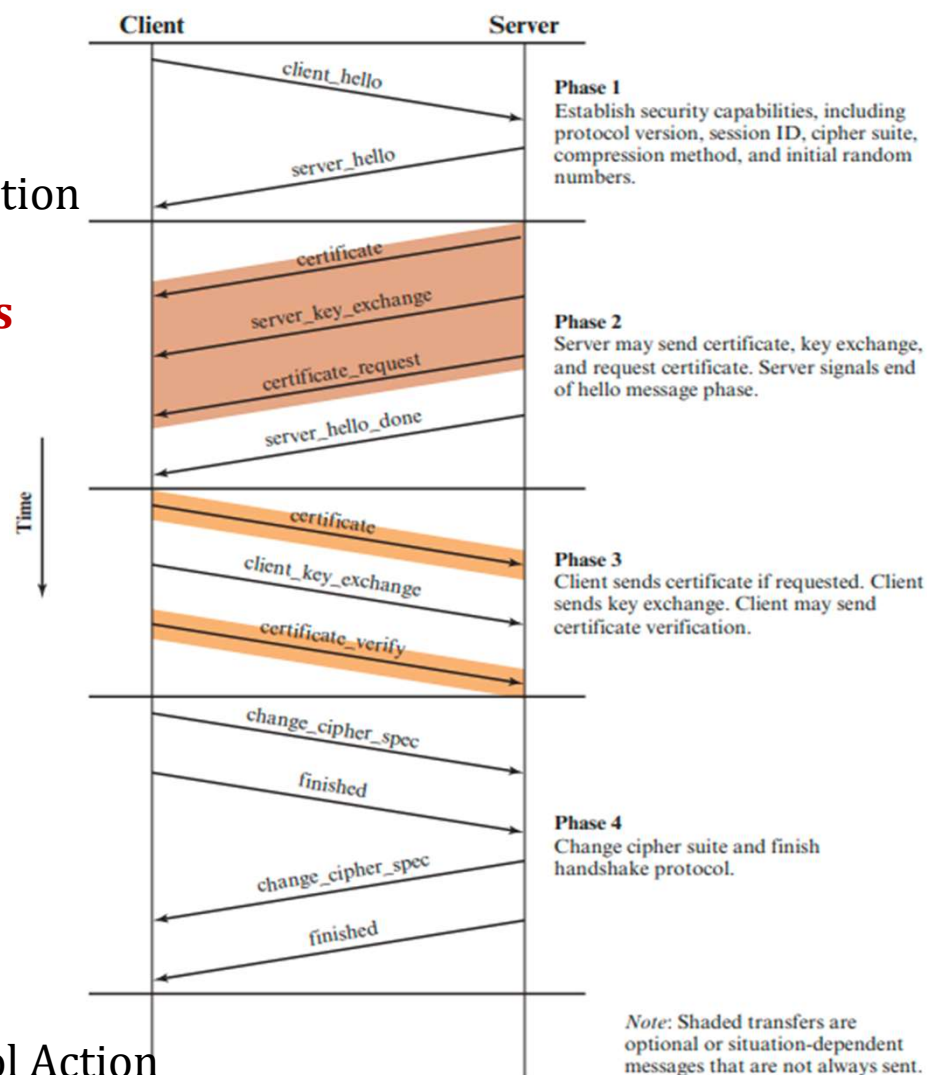


Figure 6: Handshake Protocol Action